T8 - Tkinter (legacy, low priority)

LOW PRIORITY - NOT ASSESSED ANY MORE. Only for past papers 2021 / 2022 / 2023. If time is short, skip this document completely.

Covers: building a window with `tkinter` (Label, Entry, Button, Listbox, Text), `grid`, `StringVar`, button callbacks and `messagebox`. Earned the GUI marks in the 2021-2023 Python tasks (4-5 marks of "show this screen"). The 2024 paper has no GUI.

1. What the exam asked

All three old Python parts were "GUI + database" tasks. The GUI part was always the first few marks; the database logic (see the SQLite document) earned the rest.

Paper	Screen	GUI marks	Widgets
2021, Task 1a	"Registration Page": username + password, Save, Clear, Display	**4** ("display the following screen")	2 Label, 2 Entry, 3 Button, 1 Label, 1 Text/Listbox
2022, Task 1a	"Simple Journey Fare Calculator": age Entry; Save Age, Clear All; Display Age, Display ticket price, Display Cost + two Listboxes and a total box	**4**	Label, Entry, 5 Button, 2 Listbox, 1 Text/Entry
2023, Task 1a	"Library Management System": ISBN, Title, Author, Year; Save, Clear, Display Books, display area, question box, Ask Librarian	**5**	4 Label + 4 Entry, 4 Button, display Label, question Entry

Typical extra behaviours: **Clear** empties *all* widgets (2 marks, every paper); a pop-up message on success or error (2021 duplicate user: warning; 2021 weak password: warning; 2023 `showinfo` "Book added successfully!", `showerror` "This ISBN already exists!", "Invalid year of publication!"); display all rows in a Listbox/label.

The original lecture slides and lab folders contain no tkinter material (searched all .py/.md/.pptx/.docx), so the template below is built from the exam screenshots. Full solved versions of the three papers are in `Practice/Solutions_py/` (2021_registration_, *2022_bus_fare_*, 2023_library_*).

2. Mental model (analogy)

A GUI is a **form on a clipboard** that you design once and then react to.

- `Tk()` = the clipboard itself (the window). `mainloop()` = **the receptionist waiting at the desk**: nothing happens until someone clicks, then the matching function runs.
- Widgets = the printed things on the form: Label = printed text, Entry = a blank line to write on, Button = a tick-box you press, Listbox/Text = a box where you list results.
- `grid(row=, column=)` = a **table**: every widget gets a cell. (`pack` = stack things in order, `place` = exact x/y. Use **grid**; never mix grid and pack in the same window.)
- `StringVar` = a **sticky note shared with the Entry**: `var.get()` reads it, `var.set("")` rubs it out.
- `command=function` = "when pressed, call this". **Pass the name, no brackets.**

3. Copy-paste snippets

Widget cheat sheet (`tk` = `import tkinter as tk`):

I want to...	Code
Window + title	`root = tk.Tk(); root.title("My App"); root.geometry("600x400")`
Text label	`tk.Label(root, text="Name:").grid(row=0, column=0)`
Entry (one line)	`name_entry = tk.Entry(root)` / with sticky note: `tk.Entry(root, textvariable=name_var)`
Read an Entry	`name_entry.get()` or `name_var.get()` (always a **string**)
Clear an Entry	`name_entry.delete(0, tk.END)` or `name_var.set("")`

I want to...	Code
Fill an Entry	`name_entry.insert(0, "Amy")`
Button	`tk.Button(root, text="Save", command=save)` (no brackets!)
Listbox: add / clear	`lb.insert(tk.END, "item")` / `lb.delete(0, tk.END)`
Text box: add / clear / read	`t.insert(tk.END, "x")` / `t.delete("1.0", tk.END)` / `t.get("1.0", tk.END)`
Change a Label later	`lbl.config(text="new text")`
Place it	`.grid(row=1, column=0, columnspan=2, padx=5, pady=5)`
Run	`root.mainloop()` as the **last line**

Pop-ups (2021 and 2023 papers):

```
from tkinter import messagebox

messagebox.showinfo("Success", "Book added successfully!")    # blue i
messagebox.showwarning("Username exists", "Choose another name")  # yellow !
messagebox.showerror("Error", "Invalid year of publication!")     # red X
```

Validation helpers from the three papers (plain functions, easy to test; call them inside your Save callback and show a message if they fail):

```
from datetime import date


def password_ok(password):
    """2021: at least 8 characters AND contains a number."""
    return len(password) >= 8 and any(ch.isdigit() for ch in password)


def year_ok(text):
    """2023: a whole number from 1450 up to this year."""
    return text.isdigit() and 1450 <= int(text) <= date.today().year


def fields_ok(title, author):
    """2023: neither field empty."""
    return title.strip() != "" and author.strip() != ""


def answer(question, books):
    """2023 librarian. books = list of (isbn, title, author, year)."""
    q = question.lower().strip().rstrip("?")
    if q.startswith("recommend me a book by "):
        who = q[len("recommend me a book by "):]
        for isbn, title, author, year in books:
            if author.lower() == who:
                return f"How about '{title}' by {author}?"
        return "Sorry, no book by that author."
    if q.startswith("when was ") and q.endswith(" published"):
        wanted = q[len("when was "):-len(" published")]
        for isbn, title, author, year in books:
            if title.lower() == wanted:
                return f"{title} was published in {year}."
    return "Sorry, I can only answer two kinds of question."


books = [("1", "The Signalman", "Charles Dickens", 1812)]
print(password_ok("password"), password_ok("pass1234"), year_ok("2023"),
      year_ok("1200"), fields_ok("", "x"))
print(answer("when was The Signalman published?", books))
print(answer("Recommend me a book by Charles Dickens", books))
```

Expected output: `False True True False False`, then `The Signalman was published in 1812.`, then `How about 'The Signalman' by Charles Dickens?`.

TIP: database lines (`sqlite3.connect`, `INSERT`, `SELECT`) go *inside* the callbacks; see the SQLite topic document. Replace the `ages` list in the template below with a table, and `ages.clear()` with `DELETE FROM ...`.

4. Full template, line by line (2022 "Fare Calculator" layout)

Runs as is; tested by building the window, pressing every button with `invoke()` and checking the boxes. With ages 2, 78, 68, 45, 2 it gives prices 0, 13, 13, 18, 0 and total 44, exactly the 2022 screenshot.

```
import tkinter as tk                    # the GUI library (built in)
from tkinter import messagebox          # pop-up boxes live in a submodule

ages = []                  # stands in for the database (see SQLite doc)


def ticket_price(age):
    """Price rule from the 2022 paper."""
    if age <= 2:                        # 2 and under: free
        return 0
    elif age <= 12:                     # 3 to 12: child
        return 7
    elif age >= 66:                     # 66 and over: senior
        return 13
    return 18                           # everyone else


def save_age():                         # runs when "Save Age" is pressed
    text = age_var.get().strip()        # read the Entry via StringVar
    try:
        age = int(text)                 # Entry text is a string: convert
        if age < 0:
            raise ValueError            # treat negatives as bad input too
    except ValueError:                  # int("abc") lands here
        messagebox.showerror("Error", "Please enter a whole age, 0 or more")
        return                          # stop: do not save bad input
    ages.append(age)                    # "save": would be an INSERT
    age_var.set("")                     # clear the entry for next person
    messagebox.showinfo("Saved", f"Age {age} saved")


def display_age():
    age_list.delete(0, tk.END)          # empty the Listbox first (no repeats)
    for age in ages:
        age_list.insert(tk.END, age)    # add one line per age


def display_price():
    price_list.delete(0, tk.END)
    for age in ages:                    # same order, so row n matches row n
        price_list.insert(tk.END, ticket_price(age))


def display_cost():
    total = sum(ticket_price(age) for age in ages)
    cost_text.delete("1.0", tk.END)     # Text uses "1.0", Entry uses 0
    cost_text.insert(tk.END, str(total))


def clear_all():
    ages.clear()                        # (and DELETE FROM table for a DB)
    age_var.set("")
    age_list.delete(0, tk.END)
    price_list.delete(0, tk.END)
    cost_text.delete("1.0", tk.END)


root = tk.Tk()                          # the window
root.title("Simple Journey Fare Calculator")
root.geometry("640x420")                # width x height in pixels

age_var = tk.StringVar()                # sticky note tied to the Entry
tk.Label(root, text="Enter the client's age, one at a time").grid(
    row=0, column=0, padx=10, pady=10)
```

```
tk.Entry(root, textvariable=age_var).grid(row=0, column=1, columnspan=2)

save_btn = tk.Button(root, text="Save Age", command=save_age)
save_btn.grid(row=1, column=1, pady=10)          # command=NAME, no ()
tk.Button(root, text="Clear All", command=clear_all).grid(row=1, column=2)

tk.Button(root, text="Display Age", command=display_age).grid(
    row=3, column=0)
tk.Label(root, text="Client's age in years:").grid(row=2, column=1)
age_list = tk.Listbox(root, height=6, width=14)
age_list.grid(row=3, column=1)

tk.Button(root, text="Display ticket price", command=display_price).grid(
    row=3, column=2)
tk.Label(root, text="Price per client (GBP):").grid(row=2, column=3)
price_list = tk.Listbox(root, height=6, width=14)
price_list.grid(row=3, column=3)

tk.Button(root, text="Display Cost", command=display_cost).grid(
    row=5, column=0)
tk.Label(root, text="Total cost for the journey:").grid(row=4, column=1)
cost_text = tk.Text(root, height=2, width=30)
cost_text.grid(row=5, column=1, columnspan=2)

root.mainloop()                        # hand control to the receptionist
```

Key points for the 4 layout marks: every widget in the screenshot exists, labels read the same, buttons sit where the picture shows them (adjust `row`/`column`). **Build the screen first** and run it; add logic after.

5. Traps & debugging

Mistake	What you see	Fix
`command=save()`	Function runs once at start; button does nothing later	`command=save` (no brackets)
Mixing `pack` and `grid` in one window	`_tkinter.TclError: cannot use geometry manager grid inside . which already has slaves managed by pack`	Use one manager per container
`tk.Label(...).grid(...)` stored in a variable	Variable is `None` (`.grid()` returns None)	Create first, call `.grid()` on the next line
`entry.delete(0, END)` with no import of `END`	`NameError: name 'END' is not defined`	`tk.END` (or `from tkinter import *`)
Text widget cleared with `delete(0, END)`	`TclError: bad text index "0"`	`delete("1.0", tk.END)`
Entry value used as a number	`TypeError: '<=' not supported between 'str' and 'int'`	`int(entry.get())` inside `try/except ValueError`
`mainloop()` before building widgets	Empty window, or widgets never appear	`mainloop()` last
Listbox repeats old items	Every Display press adds the list again	`delete(0, tk.END)` first
Callback uses a variable made inside another function	`NameError`	Create widgets at module level (as above)

6. Anticipated exam questions (old-paper style)

1. **[4] Display the screen shown.** Section 4 layout: Tk, title, Labels, Entry, Buttons, Listboxes/Text with `grid`.
2. **[2] Clear all windows when "Clear" is pressed.** `entry.delete(0, tk.END)`, `listbox.delete(0, tk.END)`, `text.delete("1.0", tk.END)` for every widget.
3. **[2-4] Show a message on save / on duplicate / on bad input.** `messagebox.showinfo` / `showwarning` / `showerror`.

4. **[6] Reject passwords under 8 characters or without a digit.** `password_ok()` from section 3, then `showwarning` and `return`.
5. **[3] Display everything saved.** `delete(0, tk.END)`, loop over the fetched rows, `insert(tk.END, ...)`.
6. **[7] Simulated chat answering two sentence shapes.** `answer()` from section 3, then `lbl.config(text=answer(q_entry.get(), rows))`.

7. Quick-reference box

```
import tkinter as tk; from tkinter import messagebox
root = tk.Tk(); root.title("T"); root.geometry("600x400")
v = tk.StringVar(); v.get(); v.set("")
tk.Label(root, text="Name").grid(row=0, column=0)
e = tk.Entry(root, textvariable=v); e.grid(row=0, column=1)
b = tk.Button(root, text="Save", command=save)   # NO brackets
lb = tk.Listbox(root); lb.insert(tk.END, x); lb.delete(0, tk.END)
t = tk.Text(root); t.insert(tk.END, s); t.delete("1.0", tk.END)
e.get(); e.delete(0, tk.END); e.insert(0, "s"); lbl.config(text="s")
messagebox.showinfo / showwarning / showerror ("Title", "Message")
grid(row, column, columnspan, padx, pady) - never mix with pack
root.mainloop()   # LAST line
```